

LINEAR SEARCH

Date : 10 / 07 / 2025

Topic : Linear Search

1. INTRODUCTION

Linear Search is the simplest searching algorithm. It checks each element of the list (or array) one by one from the beginning until the desired element is found or the entire list has been searched.



Key Characteristics

- Simple to understand and implement
- Works on both sorted and unsorted data
- Efficient for small datasets
- Not efficient for large datasets

2. IDEA / INTUITION

- Start from the first element.
- Compare it with the target element.
- If it matches, return the position.
- If it doesn't match, move to the next element.
- Repeat until the element is found or the end of the list is reached.

4. PSEUDOCODE

```
LinearSearch(arr, key)
n ← length(arr)
for i ← 0 to n-1           // Traverse the array
    if arr[i] == key
        | return i         // Key found at index i
return -1                  // Key not found
```

6. ACTUAL CODE (C)

```
#include <stdio.h>
int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key)
            | return i;     // key found
    }
    return -1;             // key not found
}

int main() {
    int arr[] = {15, 9, 23, 7, 31, 11};
    int n = sizeof(arr) / sizeof(arr[0]);
    int key = 7;
    int result = linearSearch(arr, n, key);
    if (result != -1)
        printf("Key %d found at index %d\n", key, result);
    else
        printf("Key %d not found in the array.\n", key);
    return 0;
}
```

Time Complexity

Best Case : $O(1)$
Worst Case : $O(n)$
Space : $O(1)$

3. ALGORITHM

1. Start from the first element (index 0).
2. Compare the current element with key.
3. If current element == key, return its index.
4. Move to the next element.
5. Repeat steps 2-4 until key is found or end of the list is reached.
6. If key is not found, return -1.

5. EXAMPLE DRY RUN

Let the array be : [15, 9, 23, 7, 31, 11] Key = 7

Step	i (index)	arr[i]	Comparison	Result
1	0	15	15 == 7 ?	No
2	1	9	9 == 7 ?	No
3	2	23	23 == 7 ?	No
4	3	7	7 == 7 ?	Yes → Found

Hence, key 7 is found at index 3.

If key = 100 (not present)
After checking all elements, return -1.

7. COMPLEXITY ANALYSIS

- Best Case : $O(1)$ (Element found at first position)
- Average Case : $O(n/2) \approx O(n)$
- Worst Case : $O(n)$ (Element not present / found at last position)
- Space Complexity : $O(1)$ (In-place, constant space)

8. PROPERTIES

- Works on both sorted and unsorted arrays.
- Performs well for small datasets.
- Very easy to implement.
- Not efficient for large datasets.

9. ADVANTAGES

- ✓ Simple and easy to understand.
- ✓ Works on any type of data (sorted/unsorted).
- ✓ No extra memory required.
- ✓ Useful for small or linear data structures.

10. DISADVANTAGES

- ✗ Inefficient for large datasets.
- ✗ Requires checking every element in worst case.
- ✗ Slower compared to advanced searching algorithms (Binary Search, etc.).

11. APPLICATIONS

- * Searching an element in a small list.
- * When data is unsorted.
- * Used in simple programs and educational purposes.



"Simplicity is the soul of efficiency."

